

TASK 04 — API Versioning & Backward Compatibility

Step 1 — Understand API versioning

Why API Versioning is Required

API versioning

Maintaining different versions of an API so that existing clients can continue working while new clients use the updated API.

For example:

Product Service

└─ /api/v1/products

└─ /api/v2/products

Example

Version 1 — Old API

GET /api/v1/products/1

Response:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 50000  
}
```

Version 2 — New API

GET /api/v2/products/1

Response:

```
{  
  "productId": 1,  
  "productName": "Laptop",  
  "amount": 50000,  
}
```

```
"currency": "INR"
}
```

Why do we need two versions

Because:

Old Mobile App ———> V1 API

|

└─ Old response format

New Mobile App ———> V2 API

|

└─ New response format

The old mobile app continues using V1.

The new mobile app uses V2.

This prevents breaking existing clients.

Real-world example

Imagine a banking application.

The old mobile app expects:

```
{
  "accountNumber": "12345",
  "balance": 10000
}
```

The new application expects:

```
{
  "accountId": "12345",
  "availableBalance": 10000,
  "currency": "INR"
}
```

```
}
```

If the bank changes the response suddenly, the old app may fail.

Instead:

```
/api/v1/accounts/12345
```

```
/api/v2/accounts/12345
```

Both versions can work independently.

API versioning allows APIs to evolve while protecting existing clients.

What is Backward Compatibility

Backward compatibility means that existing clients continue to work even after the API is updated.

Example

Suppose your old API is:

```
GET /api/v1/products
```

Response:

```
{  
  "id": 1,  
  "name": "Laptop"  
}
```

Now you add a new field:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "category": "Electronics"  
}
```

The old client can still read:

```
{
```

```
"id": 1,  
"name": "Laptop"  
}
```

It simply ignores the new field.

This is **backward compatible**.

Real-world example

Old Client

|
▼

GET /api/v1/products

|
▼

Existing response still works

Even if the server has new functionality, the old client is not affected.

Backward compatibility is mainly about **protecting existing clients from changes in the API**.

Breaking Changes

A **breaking change** is a change that causes an existing client to stop working or behave incorrectly.

Example 1 — Removing a field

Old response

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 50000  
}
```

New response

```
{  
  "id": 1,  
  "name": "Laptop"  
}
```

The price field was removed.

If the old application uses:

```
product.getPrice();
```

it may fail because the expected field is missing.

This is a breaking change.

Example 2 — Renaming a field

Old response

```
{  
  "name": "Laptop"  
}
```

New response

```
{  
  "productName": "Laptop"  
}
```

The old client expects:

```
name
```

But the new API sends:

```
productName
```

The old client may not understand the response.

Breaking change.

Example 3 — Changing the data type

Old response

```
{  
  "price": 50000  
}
```

New response

```
{  
  "price": "50000"  
}
```

Previously, price was a number.

Now, it is a string.

Some clients may fail while converting or processing the value.

Breaking change.

Example 4 — Changing the URL

Old API

GET /api/v1/products

New API

GET /api/v2/product-list

If the old client still calls:

GET /api/v1/products

and that endpoint no longer exists, the client receives:

404 Not Found

Breaking change.

Example 5 — Changing required request fields

Old request

```
{  
  "name": "Laptop"  
}
```

New API requires:

```
{  
  "name": "Laptop",  
  "category": "Electronics"  
}
```

If old clients do not send category, the request may fail.

Breaking change.

Non-Breaking Changes

A **non-breaking change** is a change that adds or improves functionality without affecting existing clients.

Example 1 — Adding a new response field

Old response

```
{  
  "id": 1,  
  "name": "Laptop"  
}
```

New response

```
{  
  "id": 1,  
  "name": "Laptop",  
  "category": "Electronics"
```

```
}
```

Old clients can still use:

id

name

They ignore:

category

Non-breaking change.

Example 2 — Adding a new API endpoint

Existing API:

GET /api/v1/products

New endpoint:

GET /api/v1/products/search?name=Laptop

The existing API is unchanged.

Non-breaking change.

Example 3 — Adding an optional request field

Old request

```
{  
  "name": "Laptop"  
}
```

New request

```
{  
  "name": "Laptop",  
  "description": "Gaming laptop"  
}
```

If description is optional, old clients can still send:

```
{
```

```
"name": "Laptop"
}
```

The request continues to work.

Non-breaking change.

Example 4 — Improving internal implementation

Suppose you change:

Old database query



New optimized database query

But the API response remains the same.

The client does not notice the internal change.

Non-breaking change.

Breaking vs Non-Breaking Changes

Breaking Change ❌

Removing a field

Renaming a field

Changing data type

Removing an API endpoint

Changing required request fields

Changing existing behavior unexpectedly

Non-Breaking Change ✅

Adding an optional field

Adding a new endpoint

Improving internal code

Adding optional request fields

Keeping existing response format

Adding new functionality

Breaking = Old client may fail.

Non-breaking = Old client continues working.

Step 2 — Implement V1

/api/v1/products

And the expected response is:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 50000  
}
```

V1 means Version 1 of our Product API.

Think of it as the **old/existing API** that clients are already using.

Client

|

| GET /api/v1/products

↓

Product Service

|

↓

V1 Response

Our responsibility is to create this API and make sure it works.

Create the V1 DTO

ProductV1Response.java

```
package com.example.product.dto;
```

```
public class ProductV1Response {
```

```
    private Long id;
```

```
    private String name;
```

```
    private double price;
```

```
public ProductV1Response(Long id, String name, double price) {  
    this.id = id;  
    this.name = name;  
    this.price = price;  
}  
  
public Long getId() {  
    return id;  
}  
  
public String getName() {  
    return name;  
}  
  
public double getPrice() {  
    return price;  
}  
}
```

Spring Boot automatically converts the Java object into JSON because we are using `@RestController`.

Create V1 Controller

Create:

controller

└─ ProductV1Controller.java

Code

```
package com.example.product.controller;  
  
import com.example.product.dto.ProductV1Response;  
  
import org.springframework.web.bind.annotation.GetMapping;  
  
import org.springframework.web.bind.annotation.RequestMapping;
```

```
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/products")

public class ProductV1Controller {

    @GetMapping

    public ProductV1Response getProduct() {

        return new ProductV1Response(

            1L,

            "Laptop",

            50000

        );

    }

}
```

Step 3 — Create V2

/api/v2/products

and:

Add additional fields

```
{

    "id": 1,

    "name": "Laptop",

    "price": 50000,

    "category": "Electronics",

    "stock": 20,

    "status": "AVAILABLE"

}
```

Suppose existing applications are using:

`/api/v1/products`

We don't want to change their response.

Instead, we create a **new API version**:

`/api/v2/products`

Create V2 DTO

Create:

dto

└─ ProductV2Response.java

Code

```
package com.example.product.dto;
```

```
public class ProductV2Response {
```

```
    private Long id;
```

```
    private String name;
```

```
    private double price;
```

```
    private String category;
```

```
    private int stock;
```

```
    private String status;
```

```
    public ProductV2Response(
```

```
        Long id,
```

```
        String name,
```

```
        double price,
```

```
        String category,
```

```
        int stock,
```

```
        String status) {
```

```
        this.id = id;
```

```
        this.name = name;

        this.price = price;

        this.category = category;

        this.stock = stock;

        this.status = status;
    }

    public Long getId() {

        return id;
    }

    public String getName() {

        return name;
    }

    public double getPrice() {

        return price;
    }

    public String getCategory() {

        return category;
    }

    public int getStock() {

        return stock;
    }

    public String getStatus() {

        return status;
    }
}
```

different in V2?

V1 has:

- id
- name
- price

V2 has:

- id
- name
- price
- category ← NEW
- stock ← NEW
- status ← NEW

So V2 is providing **more information**.

Create V2 Controller

Create:

controller

└─ ProductV2Controller.java

Code

```
package com.example.product.controller;

import com.example.product.dto.ProductV2Response;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v2/products")

public class ProductV2Controller {
```

```
@GetMapping
public ProductV2Response getProduct() {
    return new ProductV2Response(
        1L,
        "Laptop",
        50000,
        "Electronics",
        20,
        "AVAILABLE"
    );
}
```

Step 4 — Keep V1 Working

This means after creating V2, you must **not modify or remove V1**.

V1 should remain

GET <http://localhost:8083/api/v1/products>

Response:

```
{
  "id": 1,
  "name": "Laptop",
  "price": 50000
}
```

V2 should be separate

GET <http://localhost:8083/api/v2/products>

Response:

```
{
```

```
"id": 1,  
"name": "Laptop",  
"price": 50000,  
"category": "Electronics",  
"stock": 20,  
"status": "AVAILABLE"  
}
```

Notice that V2 adds:

category

stock

status

but V1 remains unchanged.

Code

ProductV1Response.java

```
package com.example.product.dto;  
  
public class ProductV1Response {  
    private Long id;  
    private String name;  
    private double price;  
  
    public ProductV1Response(Long id, String name, double price) {  
        this.id = id;  
        this.name = name;  
        this.price = price;  
    }  
  
    public Long getId() {  
        return id;  
    }  
}
```

```
}  
  
public String getName() {  
    return name;  
}  
  
public double getPrice() {  
    return price;  
}  
}
```

ProductV1Controller.java

```
package com.example.product.controller;  
  
import com.example.product.dto.ProductV1Response;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
@RequestMapping("/api/v1/products")  
  
public class ProductV1Controller {  
    @GetMapping  
    public ProductV1Response getProduct() {  
        return new ProductV1Response(  
            1L,  
            "Laptop",  
            50000  
        );  
    }  
}
```

Now **do not change this controller** when implementing V2.

V2 Implementation

/api/v2/products

with:

```
{  
    "id": 1,  
    "name": "Laptop",  
    "price": 50000,  
    "category": "Electronics",  
    "stock": 20,  
    "status": "AVAILABLE"  
}
```

So create a separate DTO.

ProductV2Response.java

```
package com.example.product.dto;  
  
public class ProductV2Response {  
    private Long id;  
    private String name;  
    private double price;  
    private String category;  
    private int stock;  
    private String status;  
  
    public ProductV2Response(  
        Long id,  
        String name,  
        double price,  
        String category,
```

```
        int stock,  
        String status) {  
    this.id = id;  
    this.name = name;  
    this.price = price;  
    this.category = category;  
    this.stock = stock;  
    this.status = status;  
}  
  
public Long getId() {  
    return id;  
}  
  
public String getName() {  
    return name;  
}  
  
public double getPrice() {  
    return price;  
}  
  
public String getCategory() {  
    return category;  
}  
  
public int getStock() {  
    return stock;  
}  
  
public String getStatus() {  
    return status;  
}  
}
```

ProductV2Controller.java

```
package com.example.product.controller;

import com.example.product.dto.ProductV2Response;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v2/products")

public class ProductV2Controller {

    @GetMapping

    public ProductV2Response getProduct() {

        return new ProductV2Response(

            1L,

            "Laptop",

            50000,

            "Electronics",

            20,

            "AVAILABLE"

        );

    }

}
```

Test Both

Start your application on:

8083

Test V1

In Postman:

GET <http://localhost:8083/api/v1/products>

Expected:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 50000.0  
}
```

Test V2

GET <http://localhost:8083/api/v2/products>

Expected:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 50000.0,  
  "category": "Electronics",  
  "stock": 20,  
  "status": "AVAILABLE"  
}
```

Therefore, **V1 has backward compatibility.**

Step 5 — Understand Versioning Strategies

URL Versioning

[/api/v1/products](#)

[/api/v2/products](#)

Example

GET [/api/v1/products](#)

and:

GET [/api/v2/products](#)

Advantages

Very easy to understand.

The version is clearly visible:

v1

v2

Easy to test in Postman.

Easy for developers and clients to understand.

Disadvantage

The URL changes between versions.

Header Versioning

Instead of putting the version in the URL, the client sends the version in an HTTP header.

For example:

GET /api/products

API-Version: 1

For V2:

GET /api/products

API-Version: 2

The URL remains:

/api/products

but the header determines the version.

Spring Boot example

```
@GetMapping(headers = "API-Version=1")
```

```
public ProductV1Response getV1() {
```

```
    return new ProductV1Response(
```

```
        1L,
```

```
        "Laptop",
```

```

        50000
    );
}
V2:
@GetMapping(headers = "API-Version=2")
public ProductV2Response getV2() {
    return new ProductV2Response(
        1L,
        "Laptop",
        50000,
        "Electronics",
        20,
        "AVAILABLE"
    );
}

```

Then Postman would send:

API-Version: 1

or:

API-Version: 2

Query Parameter Versioning

Here the version is sent as a query parameter.

Example:

GET /api/products?version=1

V2:

GET /api/products?version=2

Spring Boot example:

@GetMapping

```
public String getProducts(  
    @RequestParam(defaultValue = "1") int version) {  
    if (version == 1) {  
        return "V1 Product Response";  
    }  
    if (version == 2) {  
        return "V2 Product Response";  
    }  
    return "Unsupported version";  
}
```

Example

/api/products?version=1

→ V1

/api/products?version=2

→ V2

Content Negotiation

This approach uses the HTTP Accept header.

For example:

GET /api/products

Accept: application/vnd.company.product.v1+json

For V2:

GET /api/products

Accept: application/vnd.company.product.v2+json

The URL stays:

/api/products

but the requested media type tells the server which version is required.

Spring Boot example

```
@GetMapping(  
    produces = "application/vnd.company.product.v1+json"  
)
```

```
public ProductV1Response getV1() {  
    return new ProductV1Response(  
        1L,  
        "Laptop",  
        50000  
    );  
}
```

V2:

```
@GetMapping(  
    produces = "application/vnd.company.product.v2+json"  
)
```

```
public ProductV2Response getV2() {  
    return new ProductV2Response(  
        1L,  
        "Laptop",  
        50000,  
        "Electronics",  
        20,  
        "AVAILABLE"  
    );  
}
```

Step 6 — Document Breaking Changes

Suppose your existing Product API is:

V1 — Old Response

GET /api/v1/products

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 55000
  },
  {
    "id": 2,
    "name": "Mobile",
    "price": 25000
  }
]
```

Now suppose the business asks us to change the response.

For V2, we decide that:

- name → productName
- Add category
- Add stock
- price is changed from a number to an object containing currency and amount

This is a **breaking change** because applications using V1 cannot simply assume the V2 response has the same structure.

V2 — New Response

GET /api/v2/products

```
[
  {
    "id": 1,
    "productName": "Laptop",
    "price": {
      "amount": 55000,
      "currency": "INR"
    },
    "category": "Electronics",
    "stock": 10
  },
  {
    "id": 2,
    "productName": "Mobile",
    "price": {
      "amount": 25000,
      "currency": "INR"
    },
    "category": "Electronics",
    "stock": 20
  }
]
```

Step 7 — Test Both APIs

/api/v1/products

/api/v2/products

V1 and V2 must work independently.

A V1 client should continue working even after V2 is introduced.

V1 response

Call:

GET http://localhost:8083/api/v1/products

Response:

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 55000
  },
  {
    "id": 2,
    "name": "Mobile",
    "price": 25000
  }
]
```

V1 uses:

name

V2 response

Call:

GET http://localhost:8083/api/v2/products

Response:

```
[
  {
    "id": 1,
    "productName": "Laptop",
    "price": 55000
  },
  {
    "id": 2,
    "productName": "Mobile",
    "price": 25000
  }
]
```

V2 uses:

productName

So the main change is:

V1		V2
----	--	----

-------	--	--

name	--->	productName
------	------	-------------

price	--->	price
-------	------	-------

id	--->	id
----	------	----

Controller code

Create:

ProductController.java

```
package com.example.product.controller;

import com.example.product.dto.ProductV1Response;
import com.example.product.dto.ProductV2Response;
import com.example.product.service.ProductService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController
@RequestMapping("/api")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    // V1 API
    @GetMapping("/v1/products")
    public List<ProductV1Response> getProductsV1() {
        return productService.getProductsV1();
    }

    // V2 API
    @GetMapping("/v2/products")
    public List<ProductV2Response> getProductsV2() {
```

```
        return productService.getProductsV2();
    }
}
```

Create V1 DTO

Create:

ProductV1Response.java

```
package com.example.product.dto;

public class ProductV1Response {
    private Long id;
    private String name;
    private double price;
    public ProductV1Response() {
    }
    public ProductV1Response(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
    public Long getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public double getPrice() {
        return price;
    }
}
```

Create V2 DTO

Create:

ProductV2Response.java

```
package com.example.product.dto;

public class ProductV2Response {

    private Long id;

    private String productName;

    private double price;

    public ProductV2Response() {

    }

    public ProductV2Response(Long id, String productName, double price) {

        this.id = id;

        this.productName = productName;

        this.price = price;

    }

    public Long getId() {

        return id;

    }

    public String getProductName() {

        return productName;

    }

    public double getPrice() {

        return price;

    }

}
```

Because we don't want V1 and V2 responses to be the same.

V1:

```
private String name;
```

V2:

```
private String productName;
```

This allows us to change the API response without breaking the old API.

Product Entity

Create:

Product.java

```
package com.example.product.entity;
```

```
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.GeneratedValue;
```

```
import jakarta.persistence.GenerationType;
```

```
import jakarta.persistence.Id;
```

```
@Entity
```

```
public class Product {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    private double price;
```

```
    public Product() {
```

```
    }
```

```
    public Product(String name, double price) {
```

```
        this.name = name;
```

```
        this.price = price;
```

```
    }
```

```

public Long getId() {
    return id;
}

public String getName() {
    return name;
}

public double getPrice() {
    return price;
}

public void setName(String name) {
    this.name = name;
}

public void setPrice(double price) {
    this.price = price;
}
}

```

Notice that the database/entity still has:

name

We don't need to rename the database column just because V2 changes the API response.

Repository

Create:

ProductRepository.java

```

package com.example.product.repository;

import com.example.product.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository
    extends JpaRepository<Product, Long> {
}

```

Service

Create: ProductService.java

```
package com.example.product.service;

import com.example.product.dto.ProductV1Response;
import com.example.product.dto.ProductV2Response;
import com.example.product.entity.Product;
import com.example.product.repository.ProductRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    // V1 response
    public List<ProductV1Response> getProductsV1() {
        return productRepository.findAll()
            .stream()
            .map(product ->
                new ProductV1Response(
                    product.getId(),
                    product.getName(),
                    product.getPrice()
                )
            )
            .toList();
    }
}
```

```

// V2 response
public List<ProductV2Response> getProductsV2() {
    return productRepository.findAll()
        .stream()
        .map(product ->
            new ProductV2Response(
                product.getId(),
                product.getName(),
                product.getPrice()
            )
        )
        .toList();
}
}

```

The service gets the same database data but converts it into different DTOs.

Test using Postman

Open Postman.

Test V1

Method:

GET

URL:

http://localhost:8083/api/v1/products

Click:

Send

Expected:

[

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 55000.0  
}  
]
```

Test V2

Method:

GET

URL:

<http://localhost:8083/api/v2/products>

Click:

Send

Expected:

```
[  
  {  
    "id": 1,  
    "productName": "Laptop",  
    "price": 55000.0  
  }  
]
```

Step 8 — Create Migration Documentation

Suppose an existing application is using:

GET /api/v1/products

and receives:

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 55000
  }
]
```

Now your team creates V2:

GET /api/v2/products

which returns:

```
[
  {
    "id": 1,
    "productName": "Laptop",
    "price": 55000
  }
]
```

Notice:

V1		V2
----	--	----

id	→	id
----	---	----

name	→	productName
------	---	-------------

price	→	price
-------	---	-------

A developer using V1 needs to know about this change.

That's why we create a **migration document**.

Create the documentation file

In your project root, create:

API-MIGRATION.md

Your project will look like:

product-service

├── src

├── pom.xml

└── API-MIGRATION.md

.md means **Markdown file**.

It is normally used for project documentation.

Add the migration information

Put this inside API-MIGRATION.md:

Product API Migration Guide

Overview

The Product API supports two versions:

- V1 - Old API

- V2 - New API

V1 is maintained for backward compatibility.

New clients should use V2.

V1 API

Endpoint

GET /api/v1/products

Response

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 55000
  }
]
```

V2 API

Endpoint

GET /api/v2/products

Response

```
[
  {
    "id": 1,
    "productName": "Laptop",
    "price": 55000
  }
]
```

Changes in V2

V1 Field	V2 Field	
-----	-----	
id	id	
name	productName	
price	price	

The main change is:

name → productName

Migration Steps

Step 1

Change the API endpoint from:

/api/v1/products

to:

/api/v2/products

Step 2

Update the response mapping.

Old:

name

New:

productName

Step 3

Update the client application.

Step 4

Test the V2 API.

Step 5

Deploy the updated client.

Backward Compatibility

Existing V1 clients can continue using:

GET /api/v1/products

V1 remains available while clients migrate to V2.

New clients should use:

GET /api/v2/products

Testing

Test V1:

GET http://localhost:8083/api/v1/products

Expected:

200 OK

Test V2:

GET http://localhost:8083/api/v2/products

Expected:

200 OK

Understand the migration process

The most important part of Step 8 is this:

Old Client

| GET /api/v1/products

↓

V1 API

↓

Response:

name

After migration:

Updated Client

|

| GET /api/v2/products

↓

V2 API

↓

Response:

productName